

Fase orientada a produto, com base SQL para multiempresa, RBAC inicial, onboarding por cliente e backlog técnico de comercialização, estruturada em cima da arquitetura já documentada para backend FastAPI, middleware, PostgreSQL/pgvector, Cloudflare e operação na Hetzner.^{[1][2]}

O que essa fase entrega

Ela adiciona três scripts SQL principais: núcleo multiempresa com `tenants`, `companies`, `environments`, `connectors` e `knowledge_bases`; RBAC inicial com `roles`, `permissions`, `users`, `user_roles` e acessos por empresa; e uma base de onboarding/auditoria com projetos, tarefas e `audit_logs`.^{[2][1]}

Também inclui scripts para aplicar a estrutura no PostgreSQL, criar tenant com projeto de onboarding, semear tarefas padrão e dois documentos de apoio: modelo de onboarding por cliente e backlog técnico para comercialização.^{[1][2]}

Diagnóstico do que isso resolve

Ela ataca exatamente os principais gaps que apareceram nos documentos: a base atual menciona tenants/empresas no backend e cabeçalhos como `x-tenant-id`, mas ainda não havia, na documentação recebida, um modelo operacional consolidado para multiempresa, RBAC e onboarding escalável.^[1] Com isso, você passa a ter uma espinha dorsal de produto para sair do cenário “implantação assistida” e evoluir para uma plataforma mais comercializável, com cadastros, governança de acesso, trilha de auditoria e onboarding repetível.^{[2][1]}

Hipóteses e validações pendentes

Hipótese funcional: a estrutura proposta é compatível com um SaaS B2B multiempresa para Protheus, mas precisa ser encaixada no modelo real já existente no backend para evitar sobreposição de tabelas, nomes ou chaves.^[1]

Hipótese técnica: os scripts SQL servem como fundação consistente, porém ainda dependem de adequação nas APIs, autenticação, filtros por `tenant_id`, consultas RAG, frontend administrativo e controle de contexto do chat para que o isolamento seja realmente ponta a ponta.^{[2][1]}

Impactos

Os impactos positivos são claros: padronização de tenant, empresa e ambiente; base para permissões por escopo; onboarding repetível; e backlog técnico mais objetivo para priorização comercial.^{[2][1]}

O principal cuidado é que a aplicação precisa passar a **usar** essas estruturas em todas as camadas; criar as tabelas no banco sem adaptar backend, middleware e frontend não garante segurança nem segregação real.^[1]

Recomendação

Minha recomendação é aplicar as melhorias, revisar aderência ao schema atual do backend e mapear onde cada nova entidade será consumida: login, admin, RAG, integração Protheus, auditoria e suporte.^{[2][1]}

Depois disso, a próxima onda natural é uma **v4** com migrações aderentes ao ORM real, endpoints administrativos para tenancy/RBAC, enforcement de autorização, seeds controlados e dashboards operacionais para consumo e SLA, inclusive preparados para Power BI.^{[1][2]}

Próximos passos

1. Rodar o `apply_v3_sql.sh` em homologação e validar conflitos com o banco atual.^{[2][1]}
2. Ajustar o backend para usar `tenant_id`, `company_id` e `env_id` em autenticação, consultas e integrações.^[1]
3. Adaptar o admin para manutenção de tenant, empresa, ambiente, conectores, usuários e perfis.^{[2][1]}
4. Executar um piloto completo com `create_tenant_onboarding.sh` e `seed_onboarding_tasks.sh` para simular um cliente real.^{[1][2]}